part2/validations.rb
```
require 'active_model'

class User
  include ActiveModel::Validations

  attr_accessor :password

  validate do
    errors.add(:base, "Don't let dad choose the password.") if password == '1234'
  end
end

user = User.new
user.password = '12345'
user.valid?        # => true

user.password = '1234'
user.valid?        # => false
```

Look at how well-decoupled the code above is. ActiveModel::Validations doesn't force you to meddle with inheritance, to worry about database-related concerns, or to manage any other complicated dependency. Just by including it, you get a complete set of validation methods without adding complexity.

Speaking of ActiveModel::Validations, I promised that I'd show you how this module adds class methods such as validate to its includer. I'll keep that promise in the next chapter.